

E05 · 모델 밖의 기억 — 검색은공짜가아니다

이 권은 본문 ch10(외부 기억과 검색 비용)을 수식과 증명 없이 다시설명한다. 앞 권은 가중치 안에 넣을 수 있는 지식의 양에 상한이 있다는것을 보였다. 상한이 있으면 답이 하나 떠오른다 — **넣지 말고 밖에두면 된다**. 이 권은 그 “밖”이 무엇이고, 밖에 둔 것을 다시 읽어 오는 데얼마가 드는지를 다룬다.

답할 질문은 둘이다. 무엇을 한 번 저장하고 무엇을 때 질의마다다시 읽는가, 그리고 그 다시 읽기의 청구서는 어떻게 생겼는가.

두 번째 질문의 답이 이 권의 전부다. 검색 비용은 하나의 수가 아니라세 **항**으로 갈라지고, 세 항은 서로 다른 자원을 서로 다른 주기로먹는다. 그중 하나만 상각으로 사라지고 둘은 끝까지 남는다. 뒤의 권들이펼칠 상각 논증의 뿌리가 이 사실 하나다.

1. 밖에 둔다는결정

이 시리즈는 상태를 세 층으로 갈라 본다. 이 권의 주인공은 세 번째 층이다.

기호 풀이 Θ (slow weights) = 배포되는 가중치 그 자체. `.safetensors` 로 굽는 것. W (fast weights) = 질의를 처리하는 동안 움직이는 고정 크기 상태. E (external store) = 모델 **밖**에 텍스트·벡터·그래프로저장되는 기억. 당신의 어휘로는 vector DB 한 벌이다. q = 질의. $\hat{y}$ = 모델이 내놓는 답 (모자는 “생성된 것”을 뜻한다). f = 읽기 함수, 즉 forward pass 전체. $\text{ret}(E, q)$ = 질의 q 로 E 를 조회하는 연산.

읽기는 언제나 이 한 줄이다.

$$\hat{y} = f(q; \Theta, W, \text{ret}(E, q)) \quad (\text{R})$$

이 식이 말하는 것. 답을 만들 때 모델이 참조할 수 있는 것은 세가지뿐이다 — 가중치에 든 것, 지금 세션에서 만들어진 것, 그리고 밖에서꺼내 온 것.

이 권이 다루는 **RAG**(retrieval-augmented generation; Lewis et al., arXiv:2005.11401)는 이 식에서 W 자리를 통째로 비운 시스템이다. test time에 움직이는 것은 파라미터가아니라 **뽀뽀 나온 레코드의 정체**뿐이다. 그래서 RAG는 이시리즈에서 영점 역할을 한다 — 가중치를 하나도 건드리지 않고 밖에 둔것만으로 어디까지 가는지를 보여 주는 사례다.

밖에 두고 무엇을 사는가. 이 논문의 답은 **용량**이다. RAG-Sequence는 Natural Questions에서 44.5 EM을 기록하는데, 파라미터 수가가장 가까운 T5-large(770M)는 같은 벤치에서 28.9 EM이고 [RAG App. G], 훨씬 큰 T5-11B가 검색 없이 내는 점수도 34.5에 그친다 [RAG Table 1]. 즉 21M개의 텍스트 레코드가 [RAG § 3] 수십억 파라미터를 더 쌓아서얻었을 지식을 대신 들고 있다.

이 비교에는 논문이 스스로 밝힌 제한이 셋 붙는다. T5 수치는 재실행이아니라 발표된 값을 옮긴 것이고 [RAG App. G], RAG 쪽 생성기는 400M급 BART-large가 상한이라 수십억 파라미터 생성기 위에서도 우위가 남는지는답하지 않았으며 [RAG § 2.3], 검색기는 헤드라인을 주장하는 바로 그 두벤치 (Natural Questions·TriviaQA)에서 검색 감독을 받은 것으로 초기화되어 있다 [RAG § 4.1].

비유 참고서를 통째로 외우는 대신 책장을 옆에 두는 것이다. 머리에 넣을 수 있는 양에는 한계가 있지만 책장은 늘리면 된다. — **비유가 깨지는 곳**: 외운 것은 꺼내는 데 시간이 들지 않지만 책장은 **매번 걸어가서 찾아펼쳐야 한다**. 그리고 그 시간은 책장이 커질 수록 늘다. 이 권의 나머지전부가 그 “걸어가서 찾아 펼치는” 시간의 청구서다.

자세한 것은 ch10 § 10.1 과 § 10.2.

2. E 는 세 가지 모양이다 — text·vector·graph

E 는 하나의 자료구조가 아니라 세 구현 형태의 총칭이다. 형태를 가르는 것은 저장 방식이 아니라 $\text{ret}(E, q)$ 가 무엇을 계산하는가다.

표 E05-1. E 의 세 구현 형태와 각각의 접근 패턴.

형태	저장하는 것	ret 이 하는 일	독자가 아는 접근 패턴
text store	원문 문자열	질의와의 어휘 겹침 (BM25 등) 또는 전체 스캔	inverted index 조회 — posting list 순차 읽기와 교집합
vector store	고정 길이 실수 벡터	질의 벡터와 가까운 상위 몇 개	고차원 벡터 배열의 무작위 접근
graph store	노드와 엣지	순회	포인터 추적

세 형태는 배타적이지 않다. 실제 시스템은 같은 레코드를 둘 이상의 형태로 동시에 색인한다. RAG 자신이 그렇다 — 저장된 것은 원문 텍스트이고 (“A key feature of our memory is that it is comprised of raw text rather distributed representations” [RAG § 5]), 검색은 그 텍스트의 embedding 위에서 일어난다.

세 형태의 공통점 셋이 이 권의 systems 논지 전체를 지탱한다. **셋중 어느 것도 GEMM 이 아니다**. 셋 다 arithmetic intensity(읽어 들인 1 byte 당 수행하는 부동소수점 연산 수)가 낮다. 그리고 셋 다 HBM 이 아니라 host DRAM 또는 NVMe 에 산다 — E 는 정의상 가속기 밖의 층이다.

시스템 모델링 관점 위치가 비용의 성질을 강제한다. 커널 최적화에서 익힌 직관 — tiling 으로 재사용률을 올려 compute-bound 로 민다 — 은 이 세 형태 어디에도 이식되지 않는다. 재사용할 것이 없기 때문이다. 한 질의가 같은 벡터를 두 번 읽지 않고, posting list 를 두 번 걸지 않는다. 더 중요한 것은 **누가하는가**다. 이 연산은 가속기가 아니라 host 가 하고, TTFT 의 앞쪽에 **직렬로** 들어간다. 질의를 embedding 하고, host 에서 후보를 걷고, 뽑힌 레코드를 다시 가속기로 보낸 뒤에야 prefill 이 시작된다. 즉 E 를 붙이는 순간 서빙 경로에 **프롬프트 길이가 아니라 저장소 크기에 비례하는** 항이 하나 생긴다. GPU 이용률 대시보드에는 이 항이 “아무 일도 안 하는 구간” 으로만 보인다.

단위 하나를 처음부터 못 박아 둔다. RAG 의 E 는 전역이고 읽기 전용이며 모든 사용자가 같은 것을 본다. 따라서 이 논문이 보고하는 저장 크기는 **corpus 당** 값이다 [RAG App. C]. 이후 권들이 다룰 시스템의 상태량은 **사용자당 · 세션당**이다. 단위가 다른 두 수를 나란히 놓으면 비교가 성립하지 않는다.

기호 풀이 C_{cap} = 상태 용량 (bytes). 이 시리즈는 용량을 언제나 아래첨자를 단 C_{cap} 으로 쓴다. 첨자 없는 맨 C 는 다음절의 **chunk** 크기 전용이다. 두 글자를 섞으면 이 권의 계산이 전부 읽히지 않는다.

자세한 것은 ch10 § 10.1.

3. RAG는 읽기만완성했다

RAG는 E 를 일급 객체로 만든 논문이고, 동시에 E 에 **누가쓰는가**를 통째로 비워 둔 논문이다. 쓰기 쪽 규칙을 보면 그 빈칸이 눈에보인다.

$$E_{k+1} = \text{wr}(E_k, \hat{c}_k), \quad \hat{c}_k = S(c_k; B_s) \quad (\text{U-E})$$

기호 풀이 wr = 기록 연산. c_k = 저장하려는 원래 문맥 (대화 한토막, 문서 한 조각). $\hat{c}_k$ = **learned context**, 즉 그문맥을 미리 처리해서 만든 산출물. $S(\cdot; B_s)$ = 질의가 오기 전에문맥을 오프라인 처리하는 절차. B_s = 그 오프라인 처리에 쓰는 계산예산 (FLOPs 또는 tokens). k = **sleep 라운드** 첨자, 곧 오프라인정리 작업의 회차다. 토큰 첨자 t 와 다른 시계다.

RAG에서는 이 식의 오른쪽 절반이 세 군데 동시에 퇴화한다. 첫째, $\hat{c}_k = c_k$ 다 — 저장되는 물건이 원문 chunk 그 자체이고 요약도파생 사실도 없다 [RAG § 3, § 5]. 둘째, 따라서 $B_s = 0$ 이다. 저장전에 어떤 추론도 하지 않는다. 셋째, wr 을 실행하는 주체가 **모델이 아니다**. 쓰기는 corpus 파이프라인이나 사람이 하고, 그것도레코드 하나를 삽입하는 것이 아니라 **index 전체를 교체**하는방식으로만 한다 [RAG § 4.5].

논문 자신의 표현으로 이 기억은 “human-readable” 이고 “human-writable” 이다 [RAG § 5]. 빠진 세 번째 성질이 **machine-writable** 이고, 이시리즈가 다루는 이후 연구 전부가 그 빈칸을 채우는 작업이다.

index 전체 교체가 실제로 작동한다는 증거는 하나 있다. 2016 년 12 월과 2018 년 12 월 사이에 재임자가 바뀐 세계 지도자 82 명을 “Who is {position}?” 형태로 물었을 때, index 와 세계가 맞으면 70%(2016 index 로 2016 지도자) 와 68%(2018/2018) 를 맞히고, 어긋 나면 12%(2018 index 로 2016 지도자) 와 4%(2016 index 로 2018 지도자) 로 떨어진 다 [RAG § 4.5]. 저장소를 갈아 끼우면 모델이 아는 세계가 통째로 갈린다는 뜻이다.

시스템 모델링 관점 이 실험이 운영에 주는 함의가 이 라인의 다른 경로들과 갈리는 지점이다. 세계 지식이 바뀔 때 RAG는 **배포 아티팩트를 건드리지 않는다**. checkpoint 를 다시 굽지 않고, 서빙 가중치를 교체하지 않으며, 사용자별 delta 가 생기지 않는다. 그래서 shared-weight batching 이 온전하다 — 배치안의 모든 요청이 같은 Θ 와 같은 index 를 쓰고, 요청마다 다른 것은프롬프트에 실려 들어오는 토큰뿐이다. 롤백 단 위도 다르다. 가중치를 고치는경로에서 갱신이 틀리면 모델을 되돌려야 하지만, 여기서는 **index 를이전 스냅샷으로 되돌리면 끝난다**. 대신 갱신 입도가 거칠다 — 한 줄을고치 려고 21M 개 레코드짜리 index 를 다시 만든다.

이 퇴화 때문에 RAG는 정작 이 시리즈의 주제인 “질의 전에 예산을 써서상태를 바꾸는 시스템”에 **해당하지 않는다** — $B_s = 0$ 이기 때문이다. 그런데도 이 권이 RAG에 한 권을 쓰는 이유는 비용 구조다. 뒤의시스템들이 B_s 를 0에서 떼어내면서 물려받는 읽기 비용이 정확히 이논문의 것이다.

용어 하나를 정리하고 넘어간다. 이 논문은 generator 가중치를 “parametric memory”, index를 “non-parametric memory”라 불렀고 [RAG § 2.2, § 2.3], 그 이분법이 문헌에 들어온 자리가 여기다. **이 시리즈는 그 어휘를쓰지 않는다**. fast weight는 파라미터의 모양을 하고 있으므로 “non-parametric”이 아니고, 배포 아티팩트에 속하지 않으므로 논문이 뜻한 “parametric”도 아니기 때문이다. 두 칸짜리 어휘로는 세 층을 셀 수없다. 자세한 것은 ch10 § 10.2.

4. chunking — 무엇을 한 덩어리로 볼 것인가

검색은 레코드 단위로 일어난다. 그러므로 corpus를 어떻게 자르느냐가검색기 성능보다 먼저 결정되는 변수다.

기호 풀이 **chunking** = corpus를 검색 단위 (레코드)로 자르는 결정. 단위하나의 크기를 C 로 쓴다. n = 저장된 레코드 수. K_{ret} = 한 질의에 회수하는 레코드 개수 (회수 폭). 회수 개수에 k 를 쓰지 않는이유는 k 가 sleep 라운드 첨자로 예약되어 있기 때문이다.

RAG의 chunking은 단순하다. 2018년 12월 Wikipedia dump의 각 문서를 겹치지않는 100-word chunk로 잘라 $n = 21\text{M}$ 개 레코드를 만든다 [RAG § 3]. 즉 $C = 100$ words이고, 이 값은 선행 연구의 관행을 그대로 따른것이다.

C 하나가 세 가지를 동시에 정하고, **세 방향이 서로 반대다**.

1. **레코드 수**. n 은 C 에 반비례한다. C 를 절반으로줄이면 색인 대상이 두 배가 되고 저장 용량 C_{cap} 도 두 배가된다.
2. **회수당 프롬프트 질량**. 질의마다 프롬프트에 얹히는 토큰 수가 $K_{\text{ret}} \cdot C$ 다. C 에 정비례한다. 이것이 § 6의 두 번째비용 항이 된다.
3. **레코드의 자족성**. C 가 작으면 문장이 경계에서 잘리고대명사의 선행사가 다른 chunk에 남는다. 검색이 정답 chunk를 뽑아도 그 chunk만으로는 답이 나오지 않는 실패가 여기서 나온다.

세 번째 항목은 논문이 보고한 실패가 아니라 chunking의 정의에서 따라나온다. RAG는 **겹치지 않는** 절단을 쓰므로 [RAG § 3] 경계에서잘린 문장을 복구할 경로가 없다. 이 실패를 계측한 논문은 이 라인에 없다.

비유 사전의 표제어 단위를 정하는 일과 같다. 항목이 너무 잘면 한 항목만읽어서는 뜻이 서지 않고, 너무 굵으면 필요 없는 설명까지 통째로 읽어야한다. — **비유가 깨지는 곳**: 사전은 사람이 **항목마다자족적이 되도록** 경계를 정하지만, 여기서는 100 words마다 기계적으로자른다. 문장 한가운데를 지나가도 그대로 자른다. 자족성은 설계된 것이아니라 우연히 얻어지는 성질이다.

여기서 systems 독자가 반드시 갈라야 할 구분이 하나 있다. C 는**당신이 아는 tile 크기**와 다르다. FlashAttention의 tile을 바꾸면 속도가바뀌지만 계산되는 함수는 글자 그대로

로 같다. chunking은 **검색 대상집합 자체를 바꾼다**. C 를 바꾸면 같은 질의에 다른 레코드가 뽑히고, 프롬프트 내용이 달라지고, 답이 달라진다. E 층에서 C 는 성능 knob 이 아니라 **semantic knob**이다.

그런데 RAG는 $C = 100$ 을 고정하고 한 번도 스왑하지 않는다. 논문이 스왑한 것은 회수 폭 K_{ret} 뿐이다 [RAG § 4.5]. § 6에서 보듯 서버 비용의 두 번째 항은 K_{ret} 과 C 의 곱에 비례하는데, 논문은 곱의 한 인자만 움직여 보고 다른 인자는 관행값에 두었다. 원문 전체에서 chunk 크기가 인쇄되는 자리는 [RAG § 3]의 한 문장뿐이고, 100 words 이외의 값이 등장하는 실험은 없다. 이후 어느 논문도 C 를 비용 축으로 놓지 않았다.

자세한 것은 ch10 § 10.3.

5. 검색은 계산 문제가 아니라 대역폭문제다

vector store의 읽기는 두 개의 encoder와 하나의 상위 K_{ret} 고르기로 완전히 결정된다.

기호 풀이 embedding = 텍스트를 고정 길이 실수 벡터로 보내는 함수. 저장 쪽 함수를 ϕ_d , 질의 쪽 함수를 ϕ_q 로 쓴다. d_e = 그 벡터의 폭 (모델 폭 d 와 구분하려고 다른 글자를 쓴다). b = 벡터 원소하나의 바이트 수 (fp32면 4, 8비트면 1).

절차는 두 줄이다. 저장할 때 모든 레코드를 ϕ_d 에 통과시켜 벡터로 바꿔 둔다. 질의가 오면 ϕ_q 로 질의를 벡터로 바꾸고, **저장된 벡터 전부와 내적을 재서 큰 것 K_{ret} 개를 고른다**. 가까움을 내적으로 재는 이 연산에는 이름이 붙어 있다 — maximum inner product search (MIPS) 다.

RAG의 기여 중 하나가 이 읽기를 **학습 가능하게** 만든 것이다. 어느 레코드가 정답인지 아무도 알려 주지 않는 데도, 과제 loss의 gradient가 검색을 통과해 ϕ_q 까지 흘러간다 [RAG § 2.4]. 그런데 두 encoder의 지위가 대칭이 아니다. ϕ_q 는 학습되고, ϕ_d 는 다른 논문에서 가져와 **동결된다** — 논문의 문장은 “we ... keep the document encoder (and index) fixed, only fine-tuning the query encoder and the BART generator” 이다 [RAG § 2.4].

동결의 이유가 E 층의 첫 번째 구조적 제약이다. ϕ_d 를 **한번 바꾸면 저장된 n 개 벡터를 전부 다시 계산해야 한다**. 읽기 함수의 두 반쪽에 서로 다른 가격표가 붙어 있는 것이다. 질의 쪽은 공짜로 바꿀 수 있고, 저장 쪽은 21M 번의 forward pass를 다시 돌려야 바꿀 수 있다.

비유 도서관의 분류 체계를 바꾸는 일이다. 검색하는 사람의 습관은 오늘 바뀌도 되지만, 서가 규칙을 바꾸면 모든 책을 다시 꽂아야 한다. — **비유가 깨지는 곳**: 도서관은 새로 들어온 책만 새 규칙으로 꽂고 옛 책은 그대로 두는 절충이 가능하다. vector store에는 그 절충이 없다. 두 규칙으로 만든 벡터가 한 index에 섞이면 둘 사이의 내적이 아무 의미도 갖지 않기 때문이다. **부분 이행이 불가능하다는 것이 이 제약의 본질이다**.

이제 검색 자체의 비용을 본다. 저장된 벡터를 전부 훑는 형태에서 구조가 가장 선명하다. 이 연산은 벡터 하나당 곱셈과 덧셈을 d_e 번씩 하고, 그러기 위해 그 벡터 $d_e \cdot b$ 바이트를 읽는다. 그러므로 읽은 1 byte 당 하는 연산은

$$I = \frac{2nd_e}{nd_e b} = \frac{2}{b} \text{ [FLOP/byte]}$$

이 식이 말하는 것. 검색의 arithmetic intensity 는 레코드수에도 벡터 폭에도 의존하지 않는다. 오직 저장 정밀도만이 정한다 —fp32 면 0.5, 8 비트면 2 FLOP/byte 다. 당신이 아는 bf16 GEMV 의 1 FLOP/byte 와 같은 자릿수이며, decode 가 bandwidth-bound 인바로 그 이유가 여기에도 그대로 적용된다.

여기에 반직관적인 따름정리가 하나 붙는다. **index** 를 압축하면 트래픽은 줄지만 **intensity** 는 올라간다. b 가 작아지기 때문이다. RAG 가 보고한 CPU 메모리 100 GB → 36 GB 압축은 [RAG App. C] index 가 차지하는 바이트를 2.8 배 줄이면서 동시에 그 위의 검색을 대역폭 한계에서 조금 떼어 놓는다. 벡터 양자화가 이 층에서 표준이 된 이유는 저장 공간하나가 아니라 두 효과의 결합이다.

실제 시스템은 전수 탐색을 하지 않는다. RAG 는 FAISS 에 Hierarchical Navigable Small World 라는 근사 알고리즘을 얹어 방문하는 벡터 수를 n 보다 훨씬 작게 줄인다 [RAG § 3]. 다만 이것이 바꾸는 것은 **트래픽의 양이지 성질이 아니다**. 방문 순서가 질의에 따라 달라지는 그래프 순회이므로 접근이 무작위다. 읽는 양은 줄고 접근 지역성은 나빠진다.

시스템 모델링 관점 이 index 가 어디에 사는지가 서빙 그림을 결정한다. RAG 는 그것을 CPU 에 둔다 — “doing Maximum Inner Product Search with FAISS is sufficiently fast on CPU, so we store document index vectors on CPU” [RAG App. C]. 무압축 약 100 GB, 압축 후 36 GB 다. 즉 GPU 가 400M 파라미터 생성기를 붙들고 있는 동안 **CPU 가 그래프를 걷는다**. 당신의 회계로 옮기면 TTFT 안에 프롬프트 길이가 아니라 **index 크기에 비례하는 host-side 항**이 하나 들어온 것이다. 이 항은 GPU 를 늘려도 줄지 않고, 배치를 키우면 오히려 CPU 쪽에서 경합한다. 그리고 **논문은 이 항을 한 번도 측정하지 않는다**. “sufficiently fast” 가 전부다.

바이트를 셀 때 구분할 것이 하나 있다. 보고된 100 GB 는 벡터 배열만의 크기가 아니다. 부록은 index 가 21M 개 벡터, 총 15.3B 개 값이라고 인쇄하는데 [RAG App. G], 그 곱을 fp32 로 저장하면 벡터 배열 자체는 약 61 GB 다. 차이는 회계 대상이다 — 보고값에는 HNSW 그래프와 저장된 원문까지 들어가고, 논문은 그것을 항목별로 쪼개지 않는다 [RAG § 3, App. C]. 아래 계산은 **벡터 배열 61 GB** 쪽으로 하고, 100 GB 를 쓰면 같은 값이 1.6 배 커진다.

자세한 것은 ch10 § 10.4.

6. 청구서는 세장이다

이 권의 심장이 이 절이다. 질의 하나가 E 층에 지불하는 값은 하나의 수가 아니라 **세 항의 합**이다.

$$C_E(q) = \underbrace{C_{\text{search}}}_{\text{① 검색 자체}} + \underbrace{C_{\text{prefill}}}_{\text{② prefill 증가}} + \underbrace{\frac{C_{\text{write}}}{N_q}}_{\text{③ 쓰기}}$$

기호 풀이 N_q = 한 번 저장한 것을 몇 개의 질의가 나눠 쓰는가. 캐시의 reuse distance 와 같은 자리의 값이다. 세 번째 항만 N_q 로 나뉘어있다는 것이 이 절의 결론 전체다.

이 식이 말하는 것. 세 항은 서로 다른 자원을 서로 다른 주기로 먹는다. 하나로 합쳐 “검색 비용” 이라 부르면 그 차이가 사라지고, 차이가 사라지면 상각 계산이 틀린다.

표 E05-2. 검색이 지불하는 세 항. 마지막 열이 이 권의 결론이다.

항	무엇인가	어느 자원	언제 내는가	N_q 로 나뉘는가
① 검색 자체	질의를 벡터로 만들고 저장소를 뒤진다	host DRAM 대역폭, CPU	질의마다	아니다
② prefill 증가	뽑힌 레코드를 프롬프트에 실어 다시 읽는다	가속기 FLOPs, KV cache 바이트	질의마다	아니다
③ 쓰기	레코드를 벡터로 바꿔 index 를 만든다	가속기 FLOPs(오프라인)	저장할 때 한 번	그렇다

② **prefill 증가**가 이 권이 소유하는 개념이다. 정의는 이렇다 — 검색 결과를 프롬프트에 실어 보냄으로써 생기는 추가 prefill 비용이며, 크기는 회수된 토큰 질량 $K_{\text{ret}} \cdot C$ 가 정하고, 발생 주기는 질의마다이며, **어떤 상각으로도 줄지 않는다.**

RAG 에서 이 항의 형태는 노골적이다. 입력과 회수된 레코드를 결합하는 방법이 “we simply concatenate them” 이고 [RAG § 2.3], 회수 폭은 open-domain QA 테스트에서 RAG-Token 15 개, RAG-Sequence 50 개다 [RAG App. A]. 질의 하나가 최대 50 개 레코드를 끌고 들어온다는 뜻이다.

비유 세 항은 창고에서 물건을 꺼내 보내는 일의 세 가지 비용과 같다. 창고에 물건을 정리해 넣는 일은 한 번이고 (③), 주문이 올 때마다 창고를 뒤지는 일과 (①) 꺼낸 물건을 포장해 트럭에 싣는 일은 (②) 매번이다. — **비유가 깨지는 곳:** 진짜 창고는 물건을 꺼내면 재고가 줄지만 E 는 읽어도 줄지 않는다. 그래서 물류에서 성립하는 “많이 팔면 재고회전으로 고정비를 갚는다” 는 논리가 여기서는 ③에만 적용되고, 매번 트럭에 싣는 ②에는 적용되지 않는다.

③ **쓰기**는 RAG 에서 corpus 파이프라인이 하는 일이다. n 개 chunk 를 동결된 ϕ_d 로 한 번씩 embedding 하고 FAISS index 를 만든다 [RAG § 3]. 저장 전에 어떤 추론도 하지 않으므로 문맥당 B_s 는 0 이고, 이 비용은 corpus 스냅샷당 한 번 지불되어 배포 수명 전체에 걸쳐 나뉜다. 논문은 이 항의 시간도 FLOPs 도 보고하지 않는다.

자세한 것은 ch10 § 10.5.

7. 손으로 해 보기 — 질의 하나의 청구서

세 항을 실제 숫자로 채워 본다. 아래 계산은 전부 이 권의 예시계산이며 논문의 보고값이 아니다. 먼저 어느 수가 어디서 오는지를 갈라둔다.

논문에서 오는 값: 레코드 수 $n = 21\text{M}$, chunk 크기 $C = 100$ words, 회수 폭 $K_{\text{ret}} = 50$, index 값 개수 15.3B , 저장 encoder $\phi_d = 110\text{M}$ 파라미터 [RAG § 3, App. A, App. G].

이 예제가 고르는 값 (논문에 없다): 벡터를 fp32 로 저장, 1 word ≈ 1.3 tokens, 생성기는 8B dense 모델, 가속기 실효 처리량 125 TFLOP/s, host DRAM 대역폭 100 GB/s, 질의 자체는 15 words.

1 단계 — 프롬프트가 얼마나 길어지는가. 회수 질량은 $50 \times 100 = 5,000$ words $\approx 6,500$ tokens 다. 질의 자체는 약 20 tokens 다. 프롬프트가 20 에서 6,520 tokens 로, **326 배**가 된다.

2 단계 — 항 ②, prefill. dense 모델의 prefill 연산량은 파라미터수와 토큰 수의 곱의 두 배쯤이다. $2 \times 8 \times 10^9 \times 6,520 \approx 104$ TFLOP. 검색 없이 질의만 넣었다면 0.32 TFLOP 이었다. 시간으로 옮기면 **0.84 초 대 2.6 밀리초**다. TTFT 가 밀리초 단위에서 초단위로 넘어 간다.

3 단계 — 항 ①, 검색. 전수 탐색의 연산량은 $2 \times 1.53 \times 10^{10} \approx 30.6$ GFLOP 뿐이다. 같은 가속기라면 0.24 밀리초, 즉 계산량으로는 무시할 수준이다. 그런데 같은 연산이 벡터 배열 61 GB 를 훑으므로 100 GB/s 메모리에서는 **0.61 초**가 걸린다. **계산 시간의 2,500 배**다. § 5 의 식이 말한 것이 숫자로 확인된다 — 검색은 FLOPs 문제가 아니라 대역폭 문제이고, 근사 알고리즘은 바로 이 0.6 초를 깎으려고 존재한다.

4 단계 — 항 ③, 쓰기. index 를 만들려면 저장된 텍스트 전체를 ϕ_d 에 한 번 통과시켜야 한다. 총량은 $21 \times 10^6 \times 130 \approx 2.73 \times 10^9$ tokens 이고, 110M 파라미터 encoder 로 계산하면 약 **600 PFLOP** 이다. 같은 가속기 한장이면 4,800 초, 곧 1.3 시간이다 (FAISS 그래프를 만드는 시간은 여기 들어있지 않고 논문도 보고하지 않는다).

5 단계 — 나눗셈 하나. ③이 질의당 prefill 아래로 내려가는 지점은 $600 \text{ PFLOP} \div 104 \text{ TFLOP} \approx 5,800$ 이다. **질의 5,800 개면 쓰기 비용이 다 갚아진다.** 배포 수명 동안 10^9 개 질의를 처리하면 ③은 질의당 0.6 GFLOP 까지 내려가 ②의 **17 만분의 1** 이 된다.

여기서 읽어야 할 것은 절대치가 아니라 **비율과 순서**다. ③은 다섯자리수 아래로 사라지고, ②는 104 TFLOP 에 그대로 남는다. N_q 를 아무리 키워도 이 그림은 바뀌지 않는다. 반대로 ②를 줄이는 방법은 K_{ret} 이나 C 를 줄이는 것뿐이고, **둘 다 품질 knob** 이다.

자세한 것은 ch10 Worked micro-example.

8. 상각이 지우지 못하는 것

세 항을 상각식에 넣으면 이 권의 결론이 나온다. 한 번 낸 비용을 그것을 나눠 쓰는 질의 수로 갚아 나가면 남는 것은 이렇다.

$$\lim_{N_q \rightarrow \infty} \left(C_{\text{search}} + C_{\text{prefill}} + \frac{C_{\text{write}}}{N_q} \right) = C_{\text{search}} + C_{\text{prefill}} > 0$$

핵심 상각은 ③만 지운다. ①과 ②는 N_q 를 아무리 키워도 남는다. 이것은 회계 방식의 선택이 아니라 주기의 정의다. ③은 저장할 때 한 번내므로 그것을 나눠 쓰는 질의 수로 나눠고, ②는 질의당 한 번 내므로 나눌것이 없다.

②를 prompt cache 로 회수할 수 있지 않은가 — 이 질문에 먼저 답해야 한다. 답은 아니다. 회수 결과는 질의마다 다르므로 프롬프트의 그 구간은 캐시 hit 이 나지 않는다. 앞에 고정 system prompt 를 두면 거기까지만 hit 이고, 회수 블록이 시작되는 지점부터 끝까지 전부 miss 다. 즉 검색을 붙이는 것은 prefix cache 를 늘려 주는 일이 아니라 **prefix cache 가 덮는 구간의 경계를 앞으로 밀어내는 일**이다. 이 시리즈가 “cache 는 같은 계산의 재사용, 검색 결과는 매번 다른 내용” 이라고 반복해 갈라 놓는 이유가 이것이다.

시스템 모델링 관점 세 항이 batching 에 거는 압력을 정리한다. 좋은 소식부터 — E 는 전역이고 Θ 는 고정이므로 배치 안의 모든 요청이 같은 가중치와 같은 index 를 쓴다. shared-weight batching 은 온전하다. 나쁜 소식은 요청하나의 유효 시퀀스 수가 1 이 아니라 K_{ret} 이라는 것이다. **continuous-batching 스케줄러가 보는 요청의 토큰 비용을 사용자의 프롬프트가 아니라 검색 하이퍼파라미터가 정한다.** 운영자가 K_{ret} 다이얼을 돌리면 재학습 없이 SLA 와 정확도를 맞바꿀 수 있다는 뜻이고, 이것은 이 층이 주는 실질적 이점이다. 다만 그 다이얼의 품질 수익은 단조가 아니다 — RAG-Sequence 는 K_{ret} 을 50 까지 올릴수록 좋아지지만 RAG-Token 은 10 에서 정점을 찍고 내려온다 [RAG Figure 3]. 비용은 단조로 늘고 품질은 그렇지 않다.

RAG 자신은 이 구조의 극단에 있다. ③이 corpus 당 한 번이라 N_q 가 사실상 배포 수명 전체의 질의 수이고, 그래서 손익분기를 묻는 것 자체가 공허하다. **상각 논증이 실제로 필요해지는 것은 ③이 문맥당 · 사용자당 항으로 옮겨간 뒤이며, 그것이 뒤의 권들이 다룰 시스템들이 한 일이다.** 그때도 ②는 그대로 남는다.

자세한 것은 ch10 § 10.5.

9. 검색기가 항상 이기지는 않는다

비용을 낼 가치가 있는지는 검색이 실제로 이기는데 달려 있다. 이 논문의 표에는 지지 않는 반례가 들어 있다.

FEVER 에서 BM25 가 학습된 검색기를 이긴다. 어휘 겹침만 보는 파라미터 없는 검색기로 갈아 끼운 쪽이 75.1 과 91.6 을 기록하고, 이 논문의 중심 기여인 학습된 검색기는 74.5 와 90.6 에 그친다 [RAG Table 6]. 논문도 인정한다 — “For FEVER, BM25 performs best” [RAG § 4.5].

“모든 과제에서 개선된다” 는 문장도 자기 표에서 성립하지 않는다. 논문은 학습된 검색이 모든 과제의 결과를 개선한다고 쓰지만 [RAG § 4.5], 같은 표의 MS-MARCO Bleu-1 에서 동결 검색기와 학습된 검색기는 둘다 49.4 로 동점이다 [RAG Table 6]. 개선이 아니라 무승부다.

한 과제군에서는 기제가 통째로 무너진다. story generation 에서는 검색기가 붕괴해 입력과 무관하게 같은 레코드를 뽑고, 그러면 생성기가 레코드를 무시하는 법을 배워 검색 없는 baseline 과 같아진다 [RAG App. H]. 논문은 이것을 부록에 수치 없이, 수정 시도 없이 적는다. 이 실패에는 이름이 있다 — **retrieval collapse** 다.

반대 방향의 사실도 함께 적는다. 회수 품질은 절대적이지 않다 — FEVER 기준으로 1 위 레코드가 정답 문서인 비율이 71%, 상위 10 개 안에 있는 비율이 90%다 [RAG § 4.4]. 그런데 Natural Questions에서는 정답이 회수된 어떤 레코드에도 없을 때조차 11.8%를 맞힌다 [RAG § 4.1]. **가중치와 저장소는 서로를 부분적으로 대체한다.**

핵심 세 사실을 비용 측에 얹으면 이 권의 마지막 결론이 나온다. 항 ①과 ②는 검색기의 품질과 무관하게 언제나 지불된다. FEVER에서는 그 비용을 내고 산 학습된 검색기가 무료 baseline에 지고, story generation에서는 지불한 비용 전부가 버려진다. **이득은 과제 의존적이고 비용은 과제의존적이지 않다.** 이 비대칭이 E 층 평가의 기본 구도다.

마지막으로 이 논문의 침묵을 기록한다. RAG는 세 항 중 어느 것도 측정하지 않는다. 지연·처리량·FLOPs·index 빌드시간이 논문 어디에도 없고, 부록이 보고하는 비용은 메모리 footprint 하나뿐이다 [RAG App. C]. 회수 폭을 바꾸면 “performance and runtime”이 영향을 받는다고 명시해 놓고도 [RAG § 4.5], 그 값을 세 번스윙한 그림은 전부 품질 측만 그린다 [RAG Figure 3]. **서빙비용을 정하는 유일한 다이얼을 돌리면서 비용 측을 한 번도 그리지 않은 것이다.** 이후 이 경로의 비용 주장은 전부 이 침묵 위에 세워진다.

자세한 것은 ch10 § 10.6.

요약

- 가중치 용량에 상한이 있으면 지식을 밖에 두면 된다. 21M 개 텍스트 레코드가 수십억 파라미터가 들고 있었을 지식을 대신 든다 (44.5 EM 대 T5-11B 의 34.5). 대가는 **매질의마다 다시 읽는 비용**이다.
- E 의 구현 형태는 text·vector·graph 셋이고, 형태를 가르는 것은 저장 방식이 아니라 읽기 연산이 무엇을 계산하는가다. 셋 다 GEMM 이 아니고, 셋 다 arithmetic intensity 가 낮으며, 셋 다 가속기 밖에 산다.
- RAG는 읽기만 완성하고 쓰기를 비웠다. 저장되는 것은 원문 그 자체 ($\hat{c} = c$), 저장 전 추론 예산은 0 ($B_s = 0$), 쓰기 주체는 모델이 아니라 사람·파이프라인이며 갱신 입도는 index 전체 교체다. 그래서 저장소를 갈아 끼우면 아는 세계가 통째로 갈린다 (70/68 대 12/4).
- chunk 크기 C 는 레코드 수와 프롬프트 질량과 레코드의 자족성을 **서로 반대 방향으로 움직이는 semantic knob**이다. tile 크기와 달리 값을 바꾸면 답이 달라진다. RAG는 100 words 를 관행값으로 고정하고 한 번도 스윙하지 않았다.
- 읽기 함수의 두 반쪽에 가격표가 다르게 붙어 있다. 질의 encoder 는 공짜로 바꿀 수 있지만 저장 encoder 를 바꾸면 n 개 벡터를 전량 재계산해야 하고, 부분 이행이 불가능하다. 그래서 RAG는 그쪽을 동결한다.
- 검색의 arithmetic intensity 는 $2/b$ 로, 레코드 수에도 벡터 폭에도 의존하지 않고 저장 정밀도만이 정한다. 검색은 계산 문제가 아니라 **대역폭 문제**이며, 예제 계산에서 계산 시간의 2,500 배가 메모리 트래픽으로 나간다.
- 검색 비용은 세 항이다 — ① 검색 자체, ② prefill 증가 ($K_{\text{ret}} \cdot C$ 토큰), ③ 쓰기. **상각의 분모를 통과하는 것은 ③뿐이고, ①과 ②는 질의마다 새로 지불된다.** 회수 결과가 매번 다르므로 prompt cache 로도 회수되지 않는다 — 검색은 prefix cache 를 늘리는 것이 아니라 그 경계를 앞으로 민다.

- 학습된 검색기가 항상 이기지는 않는다. FEVER에서는 BM25가 이기고 (75.1/91.6 대 74.5/90.6), MS-MARCO에서는 동결 검색기와 동점이며 (49.4), story generation에서는 retrieval collapse로 기제가 무너진다. **이득은 과제 의존적이고 비용은 그렇지 않다.**
- RAG는 지연 · 처리량 · FLOPs · index 빌드 시간을 하나도 보고하지 않는다. 비용 축이 비어 있는 채로 이 경로가 시작되었다.

자가 점검

- E 의 세 형태를 각각 한 문장으로 정의하고, 각 형태의 읽기가 무엇을 계산하는지 말할 수 있다.
- RAG의 쓰기 경로가 어디서 세 번 퇴화하는지 ($\hat{c} = c$, $B_s = 0$, 쓰기 주체가 모델이 아님) 짚을 수 있다.
- chunk 크기 C 와 용량 C_{cap} 을 혼동하지 않고, 회수 폭 K_{ret} 을 sleep 라운드 첨자 k 와 구분해 쓸 수 있다.
- C 를 절반으로 줄이면 무엇이 늘고 무엇이 줄고 무엇이 깨지는지 세방향으로 답할 수 있다.
- 검색 비용의 세 항을 자기 서빙 config 숫자로 채우고, N_q 로 나뉘는항이 하나뿐인 이유를 **주기의 정의**로 설명할 수 있다.
- 저장 encoder를 바꾸는 비용이 왜 질의 encoder와 비대칭인지, 왜 부분이행이 불가능한지 말할 수 있다.
- (Rosetta) “vector DB 조회 $\leftrightarrow$ 검색 연산”은 왜 완전한 대응이고, “prefix cache $\leftrightarrow$ 미리 만들어 둔 문맥”은 왜 부분적 대응인지 각각 한 문장으로 설명할 수 있다. 앞의 것은 비용까지 같은 자리이고, 뒤의 것은 재사용이 목적인다는 점만 같다.

다음 권으로

이 권은 질문 하나를 열어 둔 채 끝난다. RAG는 원문을 그대로 저장했다 — 저장 전에 아무 추론도 하지 않았고, 그래서 저장하는 예산 B_s 가 0이었다. **그 자리를 0에서 떼어내면 무엇이 되는가.** 원문 대신 요약, 사실을, 정리된 메모를 저장한다면 저장되는 물건이 달라지고 저장주기가 corpus 당에서 문맥당으로 옮겨 간다. 그때 비로소 “미리 계산해 두면 이득인가”라는 상각 질문이 공허하지 않게 된다.

그러나 그 질문에 답하려면 먼저 판정 기준이 있어야 한다. 무엇을 이 라인에 속한다고 부를 것인가, 그리고 어떤 시스템이 그냥 캐시이고 어떤 시스템이 진짜로 질의 전에 예산을 쓰는가. 다음 권이 그 판별식을 세우고, 이 권이 세운 세 항 중 ② **prefill 증가만이 끝까지 따라온다**는 사실을 그위에서 다시 만난다.